Platform channels, FFI, and Kotlin Multiplatform

Platform channels, dart:ffi, Kotlin Multiplatform

Dinu-Ștefan Rusu MEC

What you should leave with

Platform channels

MethodChannel and EventChannel: calling into native code and receiving a native stream.

dart:ffi

dart:ffi and ffigen: calling C directly, and generating the bindings instead of hand-writing them.

Kotlin Multiplatform

Kotlin Multiplatform: one module, expect/actual, consumed natively by Android and iOS.

Choosing a boundary

Channels for control, FFI for compute, and what a compile-time boundary buys over a run-time one.

Native interop options

A message

A platform channel encodes a call, hands it to the other side, and decodes the reply.

A direct call

dart:ffi jumps straight into machine code already in your address space. No encoding, no thread hop, no safety net.

A build step

Kotlin Multiplatform pays the cost once, at compile time. What ships is native code on both platforms.

Lecture 11 crossed the network. This lecture stays inside one process, then crosses to a second platform at build time instead of at run time.

1

Platform channels

MethodChannel, EventChannel, call cost

Contents Page

1 **Platform channels** 4
2 dart:ffi 14
3 Compile-time code sharing 21

Dart and platform runtimes

The Dart isolate

Your widgets, your state, your heap. Its own garbage collector, and it cannot see any other heap in the process.

The platform runtime

ART on Android, the Objective-C and Swift runtime on iOS. Every native API lives here, with its own objects.

Two more threads share the process: raster, which submits frames to the GPU, and I/O, which loads assets and decodes images. Neither one runs your widget code.

Every crossing is one of two things. Encode a message and hand it to the other side, or drop to the one thing both runtimes already agree on: the C ABI.

MethodChannel: calling from Dart

```dart
class Battery {
  static const _ch = MethodChannel(
      'dev.upb.mec/battery');
  static Future<int> level() async {
    try {
      final v = await _ch
          .invokeMethod<int>('getLevel');
      return v ?? -1;
    } on PlatformException catch (e) {
      debugPrint('failed: ${e.code}');
      return -1;
    }
  }
}
```

The channel name is just a string. Both sides must agree on it byte for byte. A typo compiles, then fails at run time.

`PlatformException` carries a code the native side chose.

`MissingPluginException` means nothing answered.

MethodChannel: answering in Kotlin

```
override fun configureFlutterEngine(
    e: FlutterEngine
) {
  super.configureFlutterEngine(e)
  val msgr = e.dartExecutor.binaryMessenger
  val ch = MethodChannel(
      msgr, "dev.upb.mec/battery")
  ch.setMethodCallHandler { call, r ->
    if (call.method == "getLevel")
      r.success(batteryLevel())
    else r.notImplemented()
  }
}
```

Registered in MainActivity.

`configureFlutterEngine` runs once, when the engine attaches. The handler stays alive for the life of that engine.

`r.notImplemented()` is what the Dart side sees as a `MissingPluginException` if it calls an unknown method.

MethodChannel: answering in Swift

```
let ch = FlutterMethodChannel(
    name: "dev.upb.mec/battery",
    binaryMessenger: messenger)
ch.setMethodCallHandler { call, r in
  guard call.method == "getLevel" else {
    r(FlutterMethodNotImplemented)
    return
  }
  r(batteryLevel())
}
```

Registered in AppDelegate.
The same channel name, the same handler shape: a `call` in, a `result` closure to answer with.
`guard` plays the role of Kotlin's `if/else`: fail fast on an unknown method.

Three platform channel pitfalls

Hot reload skips native code

Only the Dart side reloads. A change in Kotlin or Swift needs a full stop and restart of the app.

The handler is on the main thread

Do the work on another thread and call `result` later, or the native UI itself will freeze.

Channel names are not namespaced

Prefix them with your package, as `dev.upb.mec/battery` does, or two plugins will collide.

Register once, per platform, per engine. A channel with no handler registered on a platform fails with `MissingPluginException`, not a compile error.

EventChannel: a sensor stream, Dart side

```dart
class StepStream {
  static const _ch = EventChannel(
      'dev.upb.mec/steps');
  static Stream<int> steps() => _ch
      .receiveBroadcastStream()
      .map((e) => e as int);
}
StreamBuilder<int>(
  stream: StepStream.steps(),
  builder: (c, s) => Text('${s.data ?? 0}'),
);
```

No `invokeMethod`. The native side pushes a value whenever it has one; `receiveBroadcastStream` turns that into a Stream.

An EventChannel is a MethodChannel for a value that repeats. The listen and cancel handshake is handled for you.

EventChannel: a sensor stream, Kotlin side

```
class StepStreamHandler(private val mgr: SensorManager) :
    EventChannel.StreamHandler {
  private var l: SensorEventListener? = null
  override fun onListen(a: Any?, events: EventSink) {
    val sensor = mgr.getDefaultSensor(TYPE_STEP_COUNTER)
    l = object : SensorEventListener {
      override fun onSensorChanged(e: SensorEvent) =
          events.success(e.values[0].toInt())
      override fun onAccuracyChanged(s: Sensor, a: Int) {}
    }
    mgr.registerListener(l, sensor, SENSOR_DELAY_NORMAL)
  }
  override fun onCancel(a: Any?) = mgr.unregisterListener(l)
}
```

Channel call cost

Two things are being paid for.
The thread hop is roughly fixed: two queue posts, two context switches. The call is asynchronous, so it also waits behind whatever the UI thread is already doing.

The encoding is not fixed at all.
The codec walks the object graph. A small map costs nothing; a hundred thousand doubles is a different matter.

⚠ The Android trap

A Dart `List<double>` decodes into boxed objects. Send a `Float64List` instead: it passes through as one raw copy.

A channel call is a message, not a function call. Its cost scales with what you put in the message, not with how simple the call looks in Dart.

Measuring channel and FFI cost

```
// release build, on a real device
const ch = MethodChannel('bench');
const n = 10000;
for (var i = 0; i < 200; i++) {
  await ch.invokeMethod('x');   // warm up
}
var sw = Stopwatch()..start();
for (var i = 0; i < n; i++) {
  await ch.invokeMethod('x');
}
print('${sw.elapsedMicroseconds / n} us');
```

Repeat with a direct FFI call instead of `invokeMethod`.

A channel call runs microseconds to low milliseconds. An FFI call stays flat.

2

dart:ffi

Calling C, generating bindings with ffigen

Contents

		Page
1	Platform channels	4
2	**dart:ffi**	14
3	Compile-time code sharing	21

dart:ffi: calling C directly

```dart
final class Stats extends Struct {
  @Double() external double mean;
  @Double() external double peak;
}
final lib = Platform.isAndroid
    ? DynamicLibrary.open('libsig.so')
    : DynamicLibrary.process();
typedef _CSig = Void Function(
    Pointer<Double>, Int32, Pointer<Stats>);
typedef _DSig = void Function(
    Pointer<Double>, int, Pointer<Stats>);
final _analyze = lib.lookupFunction<
    _CSig, _DSig>('analyze');
```

You own the memory.
No garbage collector on the other side. Use an `Arena` or a `NativeFinalizer`.
The call blocks the isolate: run a slow one on a worker isolate.

ffigen: the C header

```c
// sig.h
typedef struct {
    double mean;
    double peak;
} Stats;

void analyze(const double *xs, int32_t n, Stats *out);
void free_stats(Stats *s);
```

Do not hand-write the struct and the lookup. ffigen reads this header and generates the Dart `Struct`, the typedefs and the lookup calls for every function in it.

ffigen: the config and the generated code

```yaml
# pubspec.yaml
dev_dependencies:
  # dart pub add dev:ffigen
  ffigen: any
ffigen:
  name: SigBindings
  output: 'lib/src/sig_bindings.dart'
  headers:
    entry-points:
      - 'src/sig.h'
```

```
dart run ffigen
```

```dart
// generated: sig_bindings.dart
final class Stats extends Struct {
  @Double() external double mean;
  @Double() external double peak;
}

class SigBindings {
  final DynamicLibrary _lib;
  SigBindings(this._lib);
  late final analyze =
      _lib.lookupFunction<_AnalyzeC,
          _AnalyzeDart>('analyze');
}
```

Calling the generated bindings

```dart
final _b = SigBindings(Platform.isAndroid
    ? DynamicLibrary.open('libsig.so')
    : DynamicLibrary.process());
(double, double) analyze(Float64List xs) {
  final arena = Arena();
  try {
    final buf = arena<Double>(xs.length)
      ..asTypedList(xs.length).setAll(0, xs);
    final out = arena<Stats>();
    _b.analyze(buf, xs.length, out);
    return (out.ref.mean, out.ref.peak);
  } finally { arena.releaseAll(); }
}
```

ffigen generated `SigBindings` and `Stats`. It did not generate the `Arena`, the copy in, or the `finally`: memory management is still yours.

Zero-copy large buffers

A 4K camera frame is roughly 33 MB, thirty times a second.

Through a platform channel it is copied at least three times: native buffer, codec buffer, Dart heap.

External typed data

A `Uint8List` that is a view onto native memory, not a copy of it. Dart reads the bytes the native side wrote. A finalizer frees them later.

Ownership transfer

`TransferableTypedData` hands a buffer to another isolate as a page remap, not a deep copy. The sender loses access, which is what makes it safe.

Texture registry

The native side renders into a GPU texture and registers it. Flutter composites it directly; the pixels never touch the Dart heap.

Data big enough to matter should not be serialized at all. Share it through a pointer, or transfer ownership of it. Do not encode it twice.

Platform channels versus dart:ffi

	Platform channels	dart:ffi
Call style	asynchronous message	synchronous direct call
Cost	encoding plus a thread hop	a function call, flat in payload
Data	copied and re-encoded	shared through a pointer
Threading	hops to the platform thread	runs on the calling isolate
Reaches	any Kotlin or Swift API	the C ABI only
Fails as	a catchable exception	a crash that takes the app down

Channels for control.
One crossing per user action: open a document, read the battery, start a scan.

FFI for compute.
Anything per-frame or per-sample: signal processing, inference, image pipelines.

3

Compile-time code sharing

expect/actual, source sets, iOS integration

Contents Page

1 Platform channels 4
2 dart:ffi 14
3 Compile-time code sharing 21

Shared logic, native UI

One shared Kotlin module: models, networking, storage, business rules.

Android keeps Jetpack Compose; iOS keeps SwiftUI, or Compose Multiplatform. `expect` / `actual` covers the handful of pieces that must differ.

Android app

Consumes the module as an ordinary Kotlin library, compiled by Kotlin/JVM.

iOS app

Consumes the same module, compiled by Kotlin/Native into a framework.

Lecture 1 named this the third strategy. Stable since November 2023, Compose Multiplatform for iOS since May 2025, already in production at McDonald’s, Netflix and Forbes.

expect and actual: common declaration

```
// commonMain: the shape, no implementation
expect class KeyValueStore(name: String) {
  fun put(key: String, value: String)
  fun get(key: String): String?
}
// ordinary Kotlin, shared as-is
class ReadingsRepository(
  val http: HttpClient,
  val store: KeyValueStore,
) {
  suspend fun latest(id: String) = runCatching {
    http.get("$BASE/$id").body<Reading>()
  }.getOrElse { store.get(id) ?: throw it }
}
```

`HttpClient` is Ktor, multiplatform already. Only the key-value store needs a platform-specific body, so only it is declared `expect`.

expect and actual: platform implementations

```
actual class KeyValueStore
  actual constructor(name: String) {
  val p = ctx.getSharedPreferences(
      name, 0)
  actual fun put(
      k: String, v: String) =
      p.edit()
          .putString(k, v).apply()
  actual fun get(k: String) =
      p.getString(k, null)
}
```

```
actual class KeyValueStore
  actual constructor(name: String) {
  val d = NSUserDefaults(
      suiteName = name)
  actual fun put(
      k: String, v: String) =
      d.setObject(v, forKey = k)
  actual fun get(k: String) =
      d.stringForKey(k)
}
```

iosMain calls NSUserDefaults directly. No bridge, ordinary Kotlin APIs on both platforms.

Source sets

```
shared/src/
  commonMain/kotlin/
  androidMain/kotlin/
  iosMain/kotlin/
androidApp/    # Compose UI
iosApp/        # Xcode, SwiftUI
```

```
// shared/build.gradle.kts
kotlin {
  androidTarget()
  iosArm64(); iosSimulatorArm64()
  sourceSets {
    commonMain.dependencies {
      implementation(libs.ktor.core)
    }
    androidMain.dependencies {
      implementation(libs.ktor.okhttp)
    }
    // iosMain: libs.ktor.darwin
  }
}
```

iOS integration

```
./gradlew \
  :shared:assembleSharedXCFramework
```

```
import Shared
struct DeviceView: View {
  @State var reading: Reading?
  var body: some View {
    Text("\(reading?.tempC ?? 0)")
      .task {
        reading = try? await repo()
      }
  }
}
```

No bridge at run time.
Kotlin classes compile to native code and appear as Objective-C classes, called the way Swift calls any framework.
`repo()` is a `ReadingsRepository`. A Kotlin `suspend` function arrives as Swift `async/await`.

Flutter and Kotlin Multiplatform

	Flutter	Kotlin Multiplatform
UI	one UI, Impeller draws both	native per platform, or Compose
Shared	everything, incl. pixels	whatever is in commonMain
Boundary	run time, per call	compile time, per build
Language	Dart for the whole app	Kotlin, plus Swift for iOS
Team	one team ships both	comfort on each platform
Maturity	stable, large plugin ecosystem	stable since 2023

Flutter crosses the boundary at run time. Kotlin Multiplatform crosses it at compile time.

Native interop in the project

Graded requirement 4 asks for gRPC, or FFI, or a platform channel.

Lecture 11 gave you gRPC. Today gave you the other option: native code called from Dart.

⇄ Reach for a platform channel

If the work is a one-shot call into an OS feature you cannot reach from Dart: a native sensor API, a permission dialog, a vendor SDK.

ϟ Reach for dart:ffi

If the work is per-sample computation you already have as C, C++ or Rust: a filter, a codec, a small inference kernel.

Pick the one your measured claim can say something about. Requirement 5 wants one number you measured. A channel call's latency or an FFI call's throughput are both defensible choices, if you show the method.

Summary

1. **Every crossing inside the process is a message or a direct call.** A platform channel encodes and hops threads; dart:ffi jumps straight into machine code you already share.
2. **Channels are for control, FFI is for compute.** Low-frequency intents versus anything on a per-frame or per-sample path.
3. **Kotlin Multiplatform moves the boundary to build time.** The shared module is native code on both platforms, not a bridge crossed on every call.

Further reading

Platform channels and FFI

docs.flutter.dev/platform-integration · dart.dev/interop/c-interop

ffigen

pub.dev/packages/ffigen

Kotlin Multiplatform

kotlinlang.org/docs/multiplatform

Before the exam. Run the channel-versus-FFI benchmark from this lecture on your own phone, in a release build.

Questions?

dinu_stefan.rusu@upb.ro · Microsoft Teams: course channel